

Optimization of Photovoltaic and Battery Storage Systems

Christian Besold Jakob Haberstock Nandini Joshi Robert Leichtl Florian Merlau

Friedrich-Alexander-Universität Erlangen-Nürnberg, Computer Science 7

July 22, 2025

-
- Introduction
 - 2. Basic Implementation
 - 3. Heatpump
 - 4. Optimized Strategy
 - 5. Electrical Vehicle
 - 6. Live Demo
 - 7. Evaluation
 - 8. Time Constraints
 - 9. Future work

The Problem: An Economic Challenge

PV systems and batteries promise greater energy independence and reduced electricity costs.

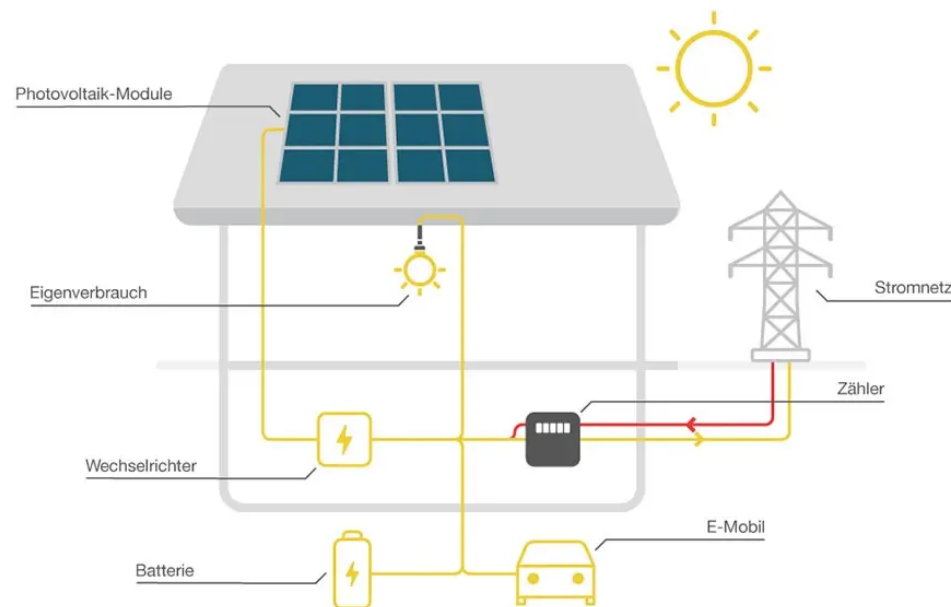


Figure: Photovoltaic modules. Source: EWE^a

^a<https://www.ewe.de/solar/ratgeber/pv-anlage>

Research Question

What is the optimal number of PV modules and battery count to **maximize the financial benefit** for a household?

Project Goals

Develop a **simulation model** that enables:

- to find the best combination for PV Count and Battery count for a household

1. Introduction

Basic Implementation

3. Heatpump

4. Optimized Strategy

5. Electrical Vehicle

6. Live Demo

7. Evaluation

8. Time Constraints

9. Future work

- **Initial Approach:**
 - Utilization of AnyLogic's **internal HSQL database**.
 - Direct data import from CSV files **via Java**.
 - Data retrieval using a **Java API** for handover to other classes in AnyLogic.

Challenges During Implementation

- **Problem with AnyLogic's Internal Database:**
 - **Faulty CSV Imports:** During testing of the implementation, errors (ArrayOutOfBoundery) occurred when importing large CSV files via Java into the HSQL database.

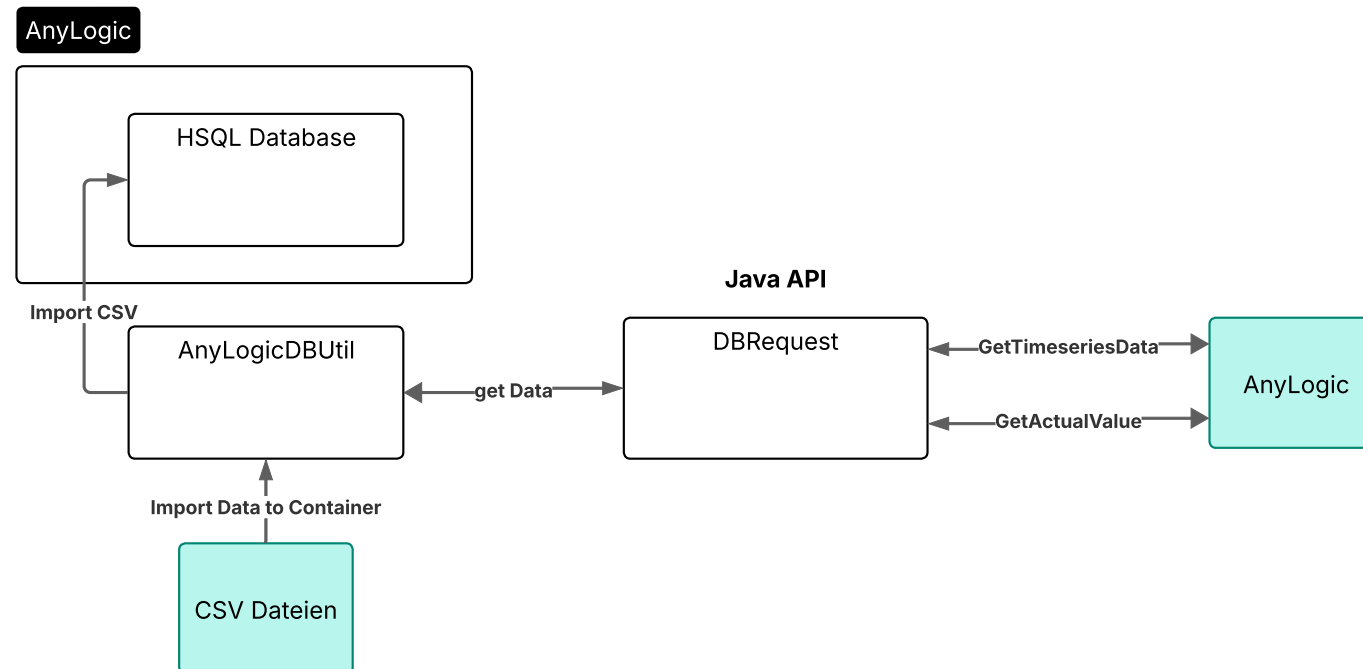


Figure: PostgreSQL database schema, own depiction

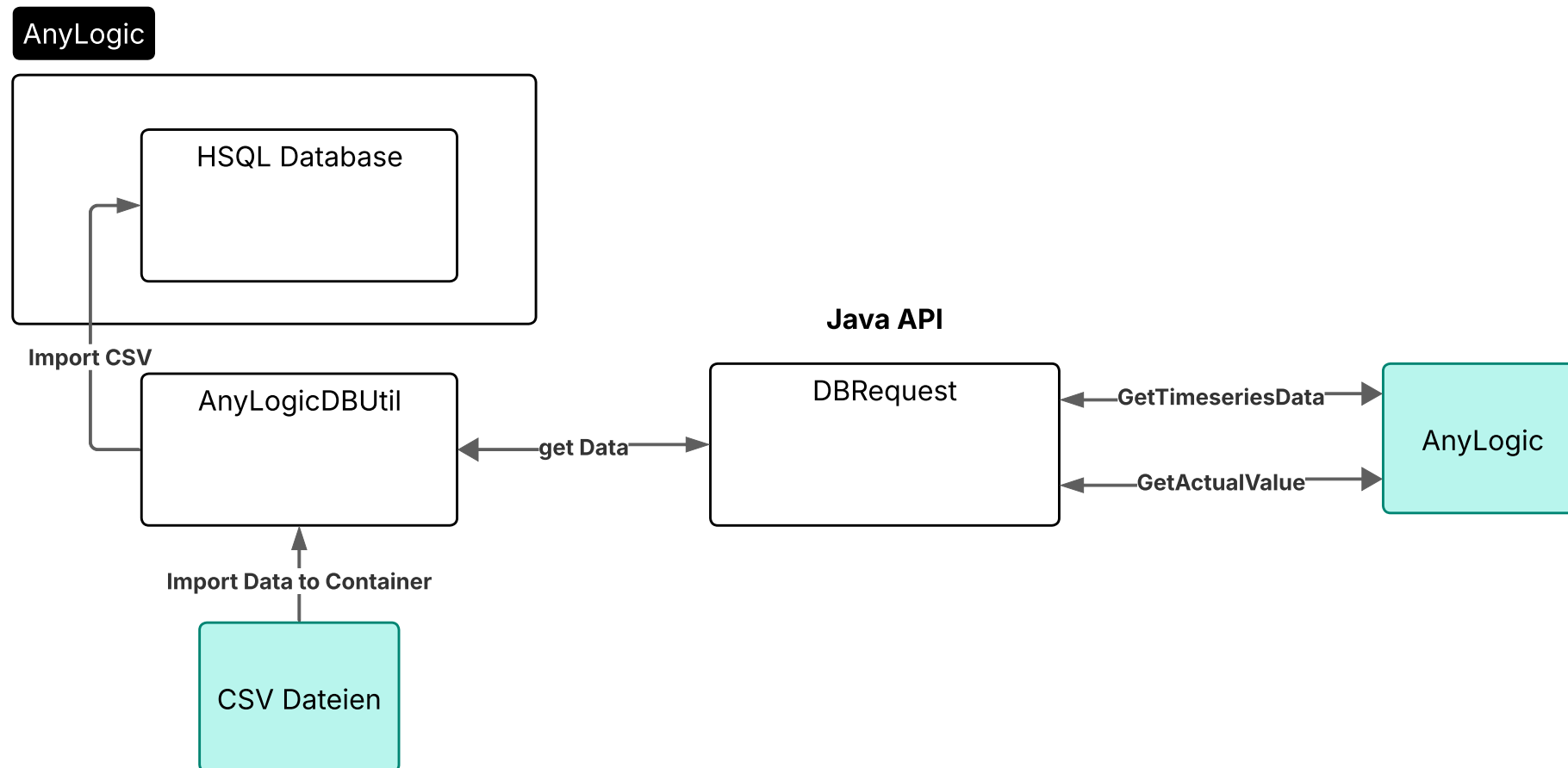


Figure: PostgreSQL database schema, own depiction

External PostgreSQL Database

Chosen Solution: Migration to an external PostgreSQL database, managed with Docker.

- **Data Import:**
 - CSV files are initially **imported into PostgreSQL using Python**.
 - **Advantage of Python:** Ideal for efficient data processing and import into SQL databases.
- **Data Retrieval:**
 - Data is still **retrieved from PostgreSQL via a Java API** and provided to AnyLogic classes.

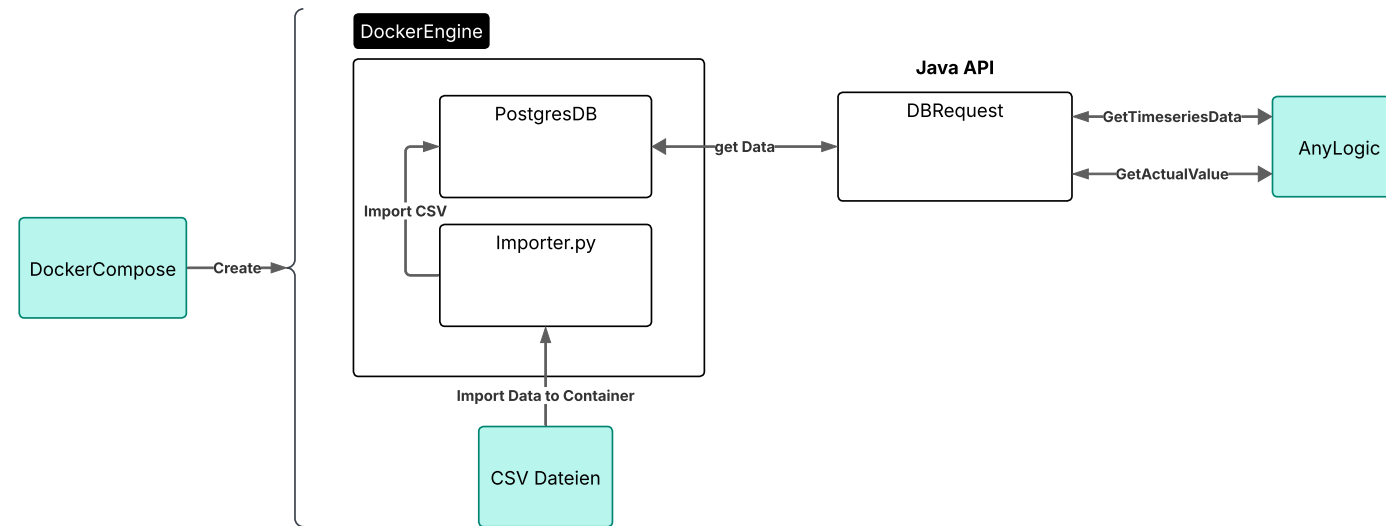
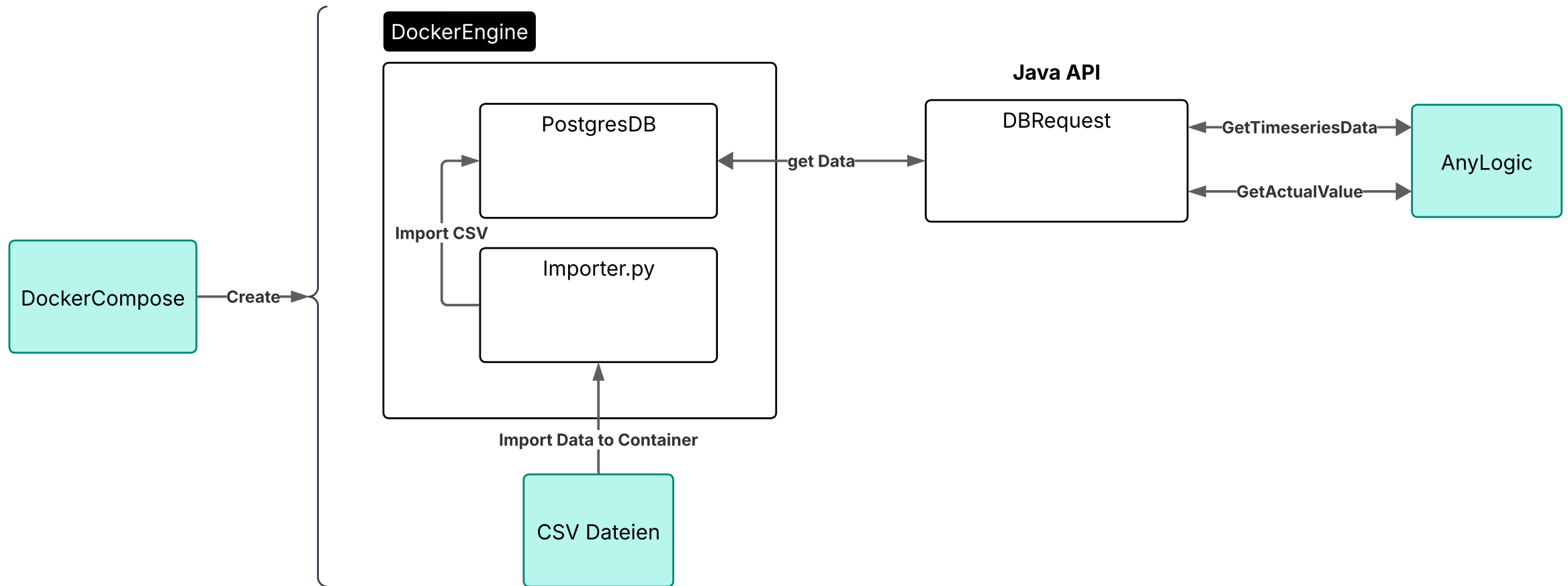


Figure: External database implementation, own depiction

Database

External PostgreSQL Database



- **Problem:** External database queries were too slow during simulation runtime (12 days simulated per real second).
- **Solution:** Introduced in-memory caching: preloaded a defined data range into RAM to allow direct access.
- **Result:** Achieved a speed-up of **45x** — now simulating 1.5 years per second.

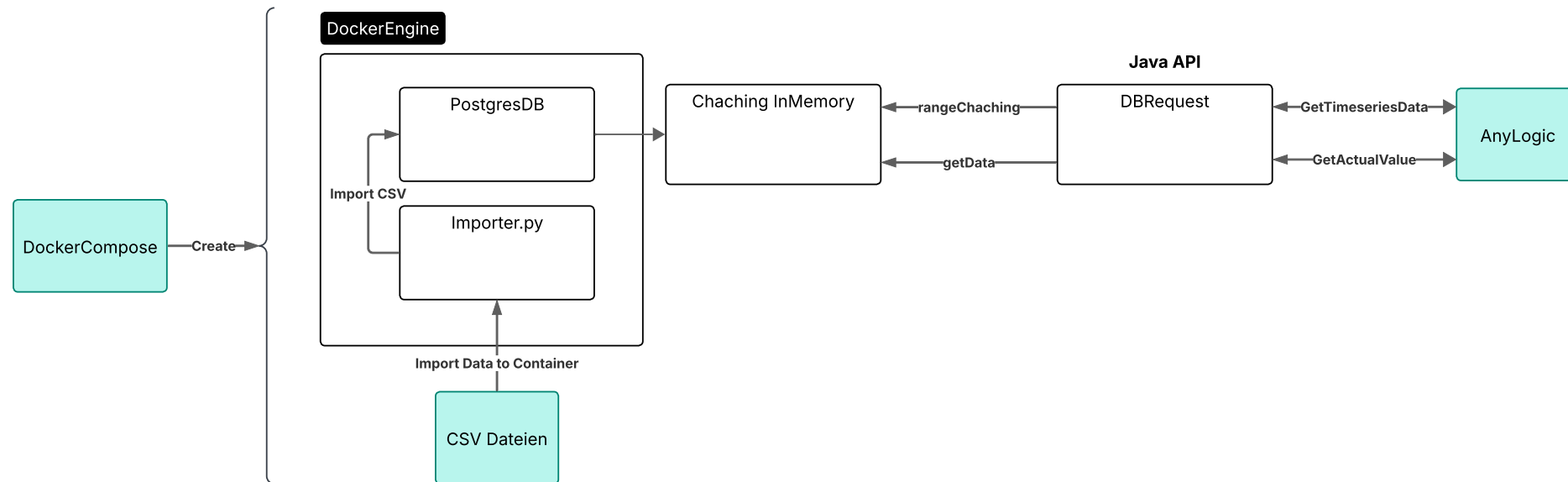


Figure: Improved performance with in-memory data access (own illustration)

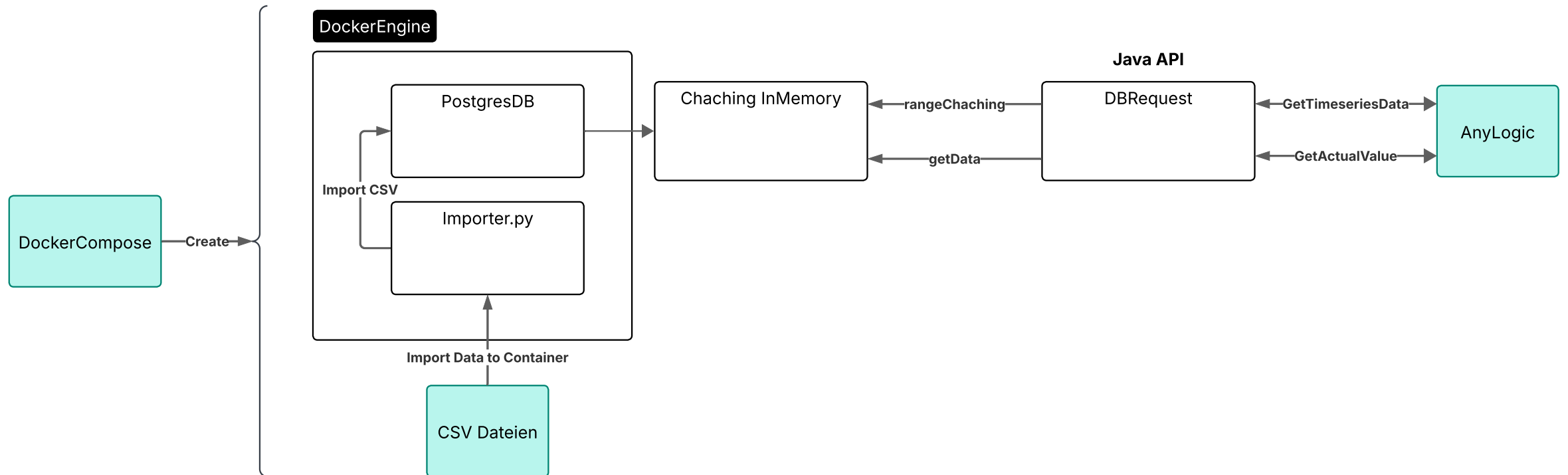


Figure: External database implementation with in-memory caching own depiction

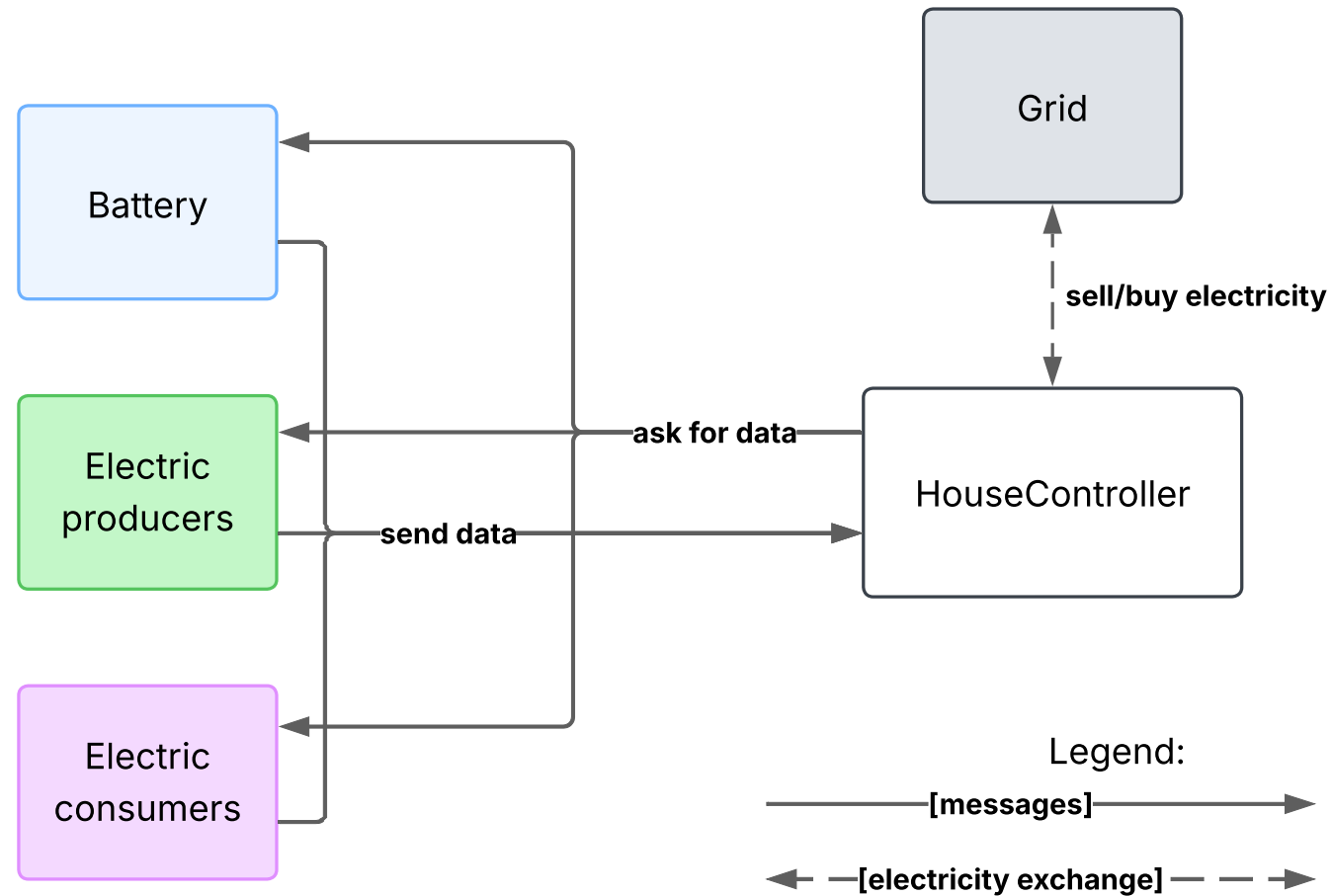


Figure: Diagram for House Controller

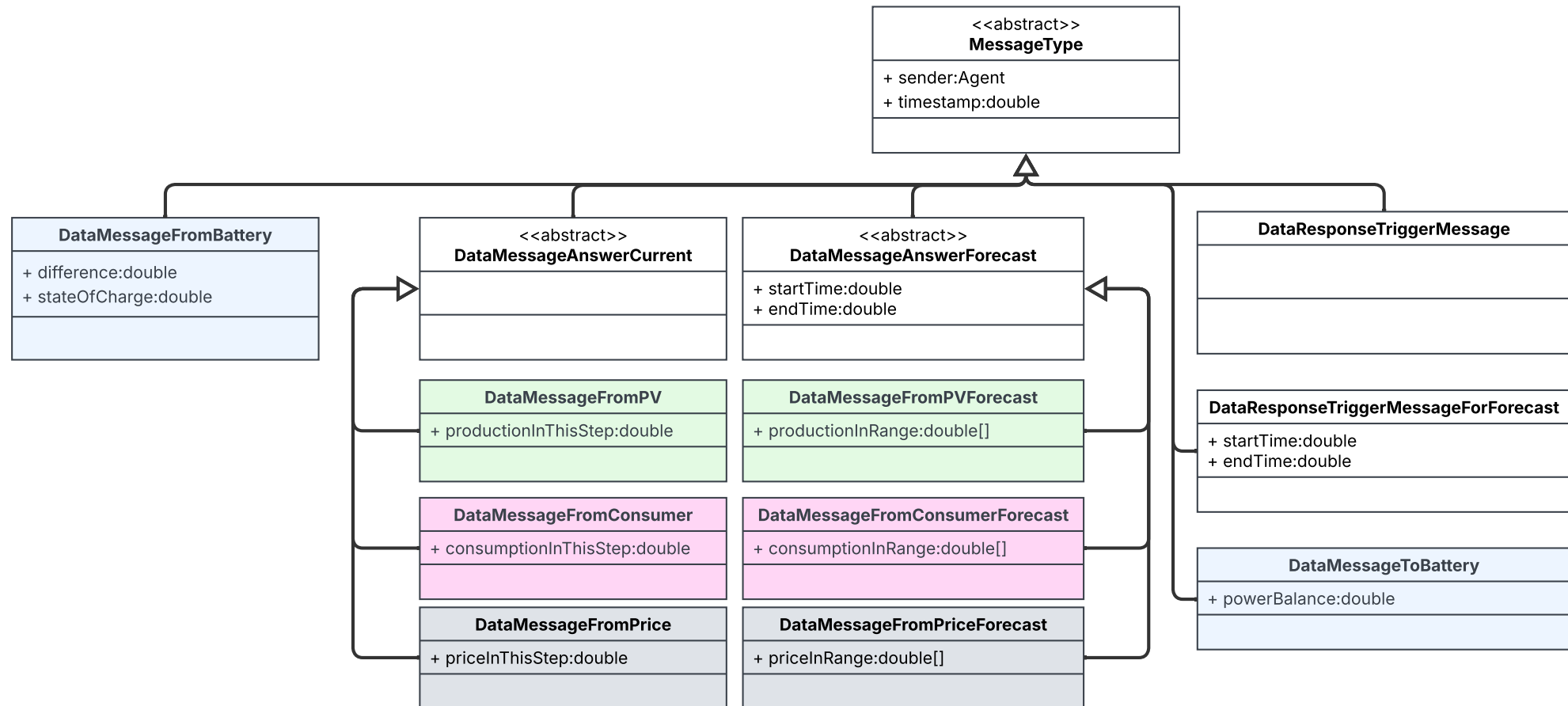
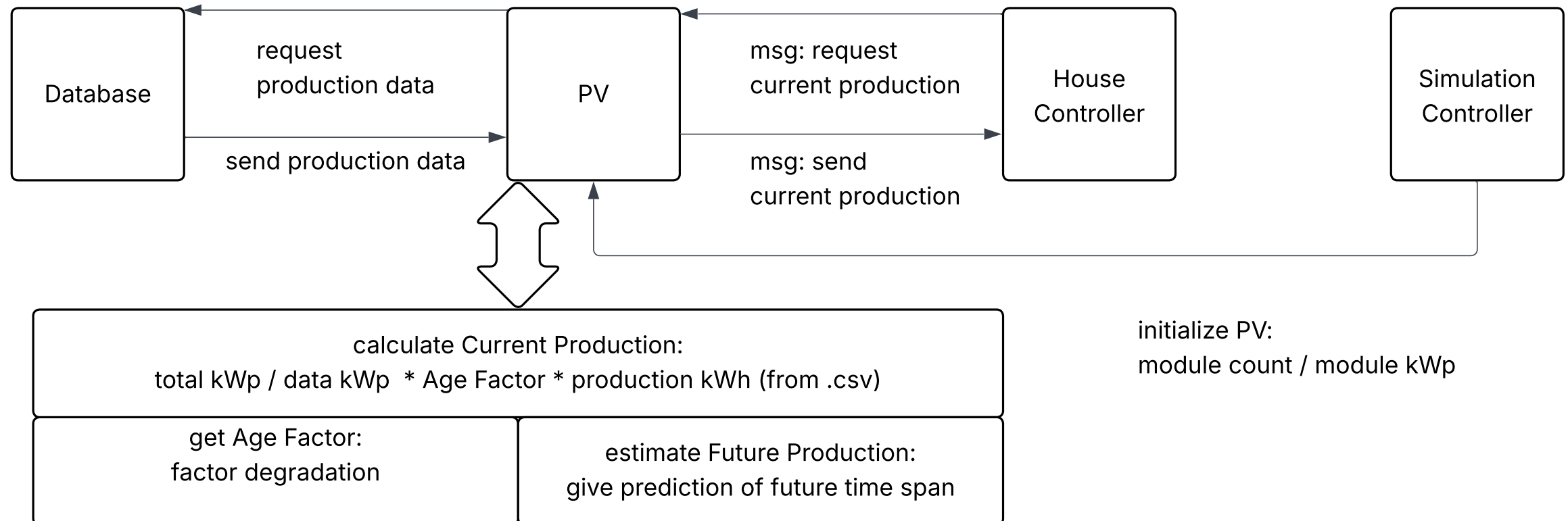


Figure: Class Diagram for Message Classes



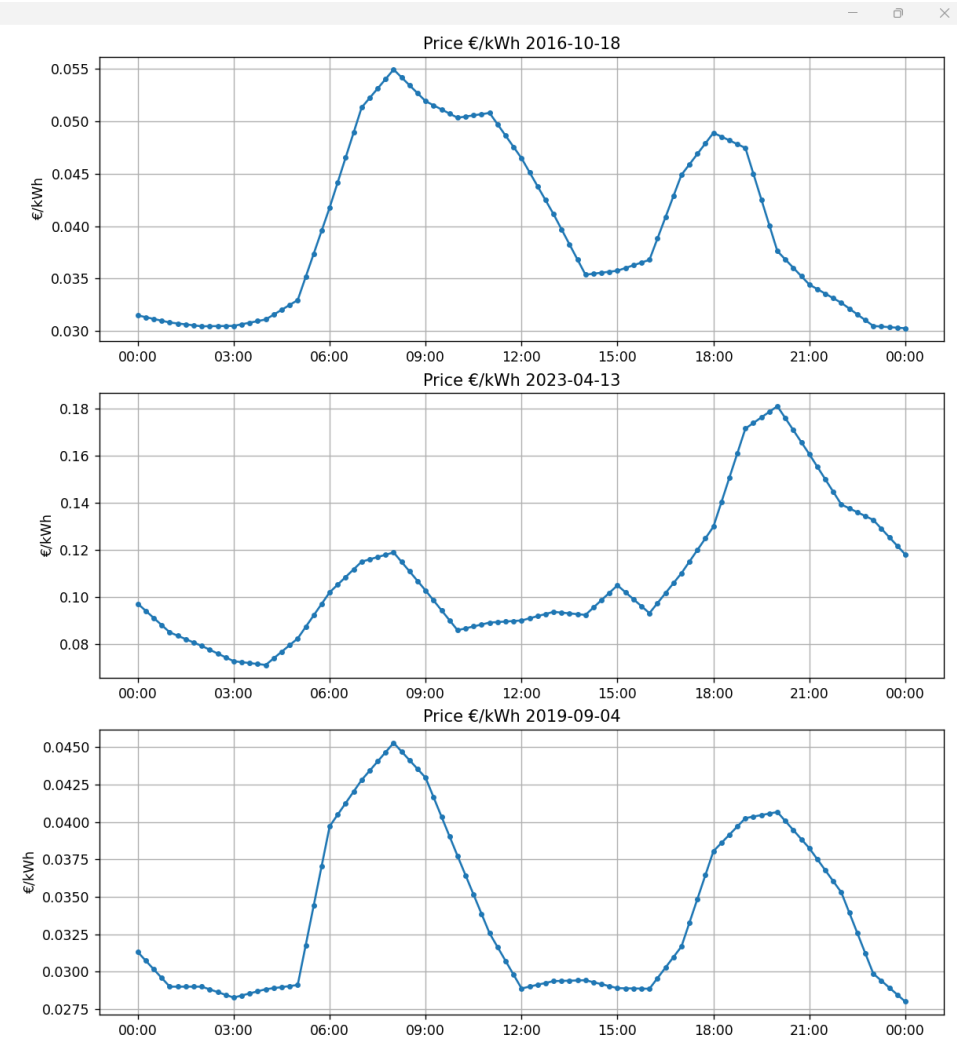
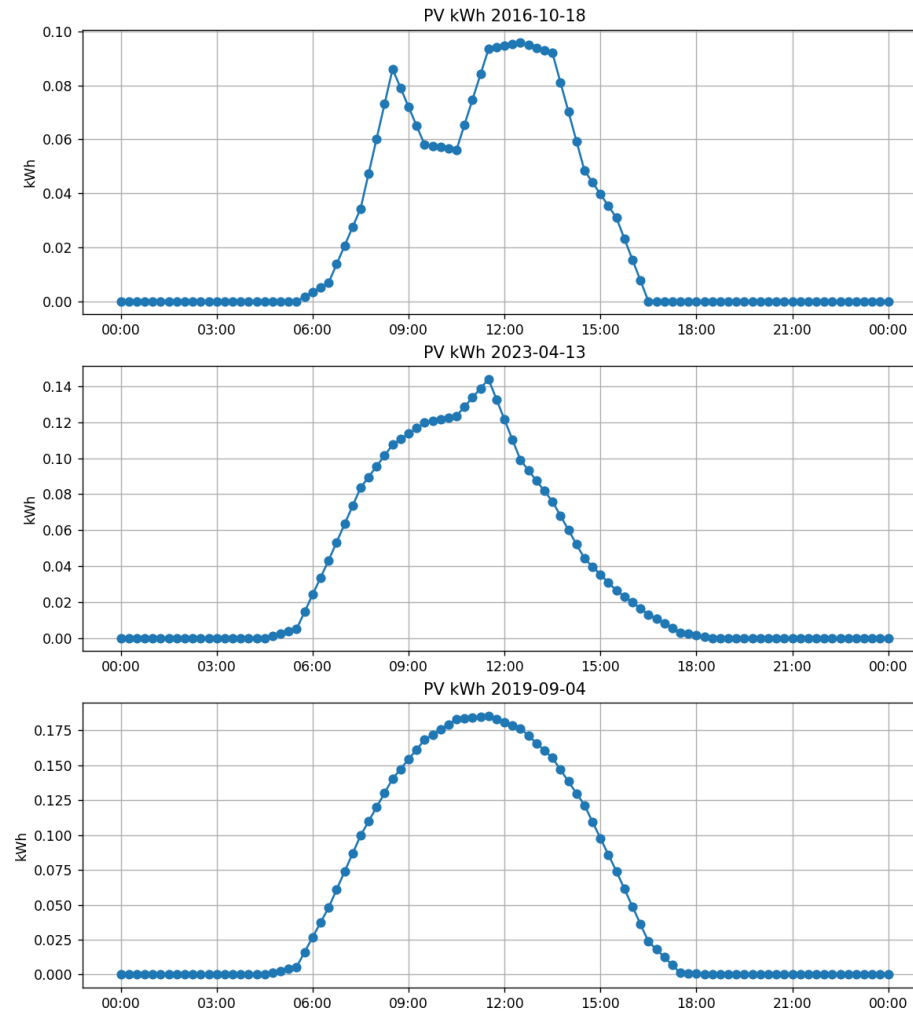
- PV source: PVGIS¹
- Reasons for choosing this PV source:
Known kWp / free choice of location / few factors needed to calculate production
- Validation Problem: PVGIS provides simulated production data
→ Result must be validated before use
- Both datasets use 1 hour intervals
→ used linear interpolation to get 15 min intervals

Time	Production (kWh)
2005-01-01 14:45:00	0.0146
2005-01-01 15:00:00	0.0097
2005-01-01 15:15:00	0.0049
2005-01-01 15:30:00	0.0000

¹https://re.jrc.ec.europa.eu/pvg_tools/en/

PV.csv and price.csv

Figure 1



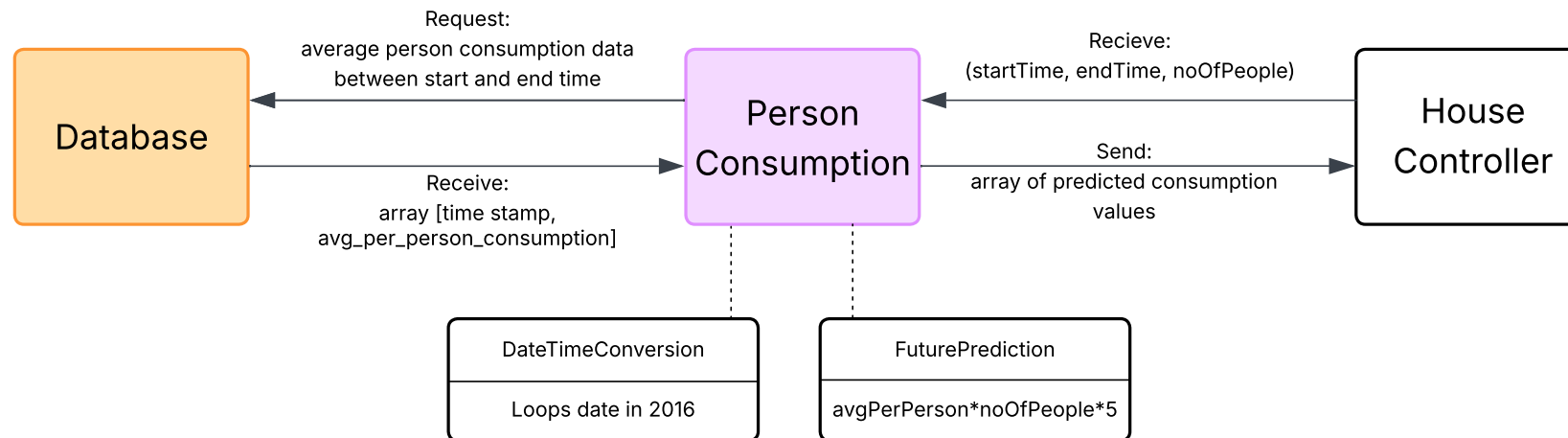
2 Options to model prices:

- 1. use standard real world contracts
- ⇒ problem: no fluctuations in prices makes optimizer lack relevancy
- 2. use wholesale prices in combination with flexible contracts to enable the optimizer
- ⇒ contract ² we use:
- $$\text{end price} = ((\text{wholesale price} * 1.03) + 1.50 \text{ cent}) * 1.20 \text{ MwSt}$$
- wholesale price source: Bundesnetzagentur³
 - equal prices for selling and buying
- ⇒ selling energy more complex and potentially less rewarding in real life

Time	Wholesale price (€/kWh)	End price (€/kWh)
2016-01-01 03:00:00	0.01681	0.03877
2016-01-01 03:15:00	0.01696	0.03896
2016-01-01 03:30:00	0.01711	0.03914
2016-01-01 03:45:00	0.01726	0.03933

²<https://www.awattar.at/tariffs/hourly>

³<https://www.smard.de/home/downloadcenter/download-marktdaten/>



Highlights:

- Accepts user defined inputs
- Requests and retrieves from the PostgreSQL database
- Calculates per-person consumption:

$$\text{avg_per_person_consumption} \times \text{number_of_people}$$

- Data Source: Kaggle - Household Load and Solar Generation⁴

utc_timestamp	average_per_person_consumption (kWh)
2016-01-01 00:00:00Z	0.000
2016-01-01 00:15:00Z	0.039
2016-01-01 00:30:00Z	0.029
2016-01-01 00:45:00Z	0.029
2016-01-01 01:00:00Z	0.046

⁴<https://www.kaggle.com/datasets/mexwell/household-load-and-solar-generation>

-
1. Introduction
 2. Basic Implementation
 - Heatpump
 4. Optimized Strategy
 5. Electrical Vehicle
 6. Live Demo
 7. Evaluation
 8. Time Constraints
 9. Future work

Heatpump

Diagram

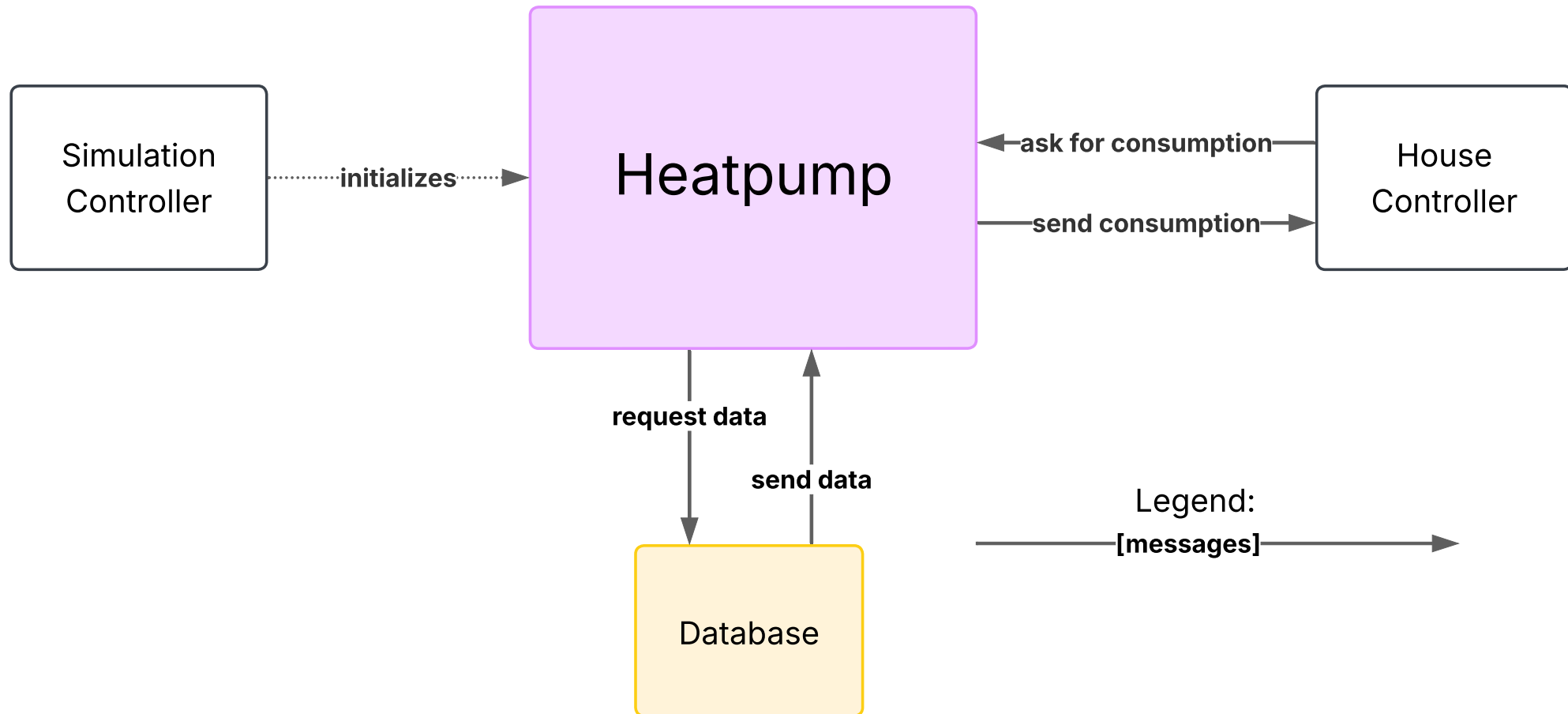


Figure: Heatpump Diagram

The Challenge

- **Limited Public Data:** No comprehensive datasets available online covering both area requirements and electrical consumption of heat pumps
- **Simulation Requirements:** Needed detailed consumption patterns for accurate modeling

Our Solution

- **Custom Model Data:** Received specialized heat pump data from Jonathan's proprietary model
- **High-Resolution Data:** Time-series consumption data with detailed temporal resolution

Simulation Parameters

- **Household Size:** 3-person household
- **House Area:** 130 square meters
- **Simulation Period:** 10 years of operation data
- **Time Resolution:** 1-hour intervals (original), blown to 15-minute intervals
- **other parameters:** Default parameters in the “default_parameters.xlsx” file

Sample Data Structure

Timestamp	Consumption (kWh)
2016-01-01T01:00:00	0.0
2016-01-01T02:00:00	0.7609
2016-01-01T03:00:00	0.7609
2016-01-01T04:00:00	0.7768

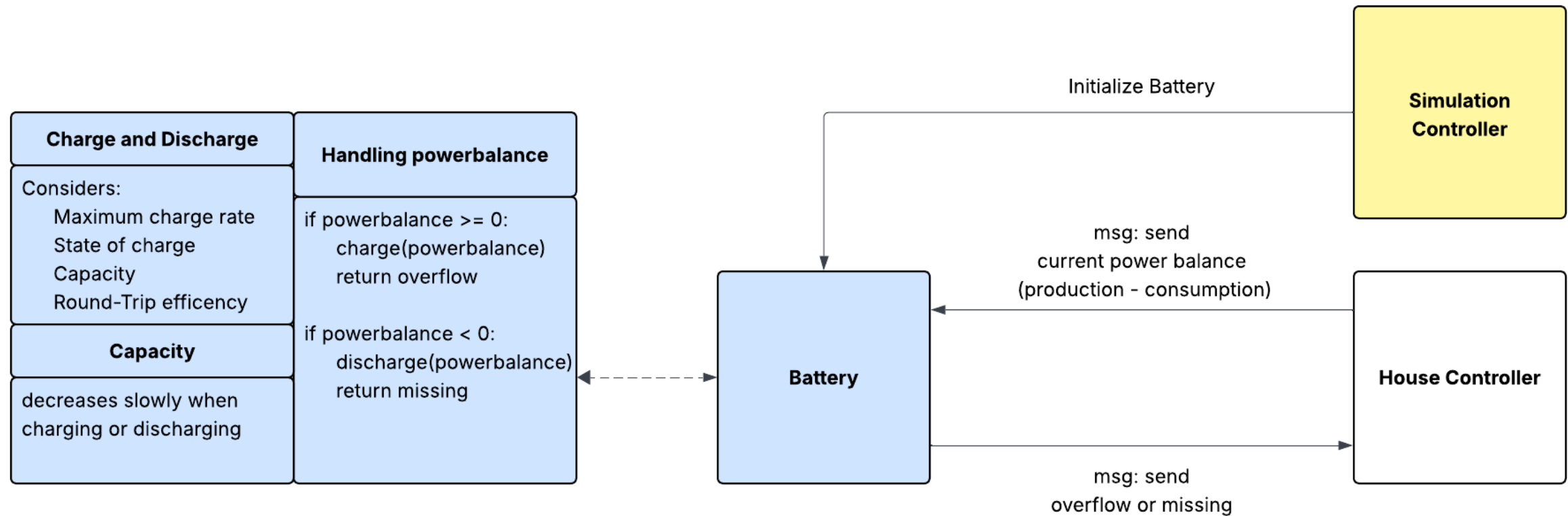


Figure: Battery

-
1. Introduction
 2. Basic Implementation
 3. Heatpump
 - Optimized Strategy
 5. Electrical Vehicle
 6. Live Demo
 7. Evaluation
 8. Time Constraints
 9. Future work

- **Variations:** Our house controller can be initialized with either a **greedy strategy** or a **strategy based on optimization**.
- **Purpose:** This enables the simulation to compare typical user behavior with an idealized upper bound—relevant for users with advanced smart meters.

Greedy Strategy

- Purchases energy when none is available; sells excess when the battery is full.
- **Simple but economically inefficient.**
- May cause higher battery degradation in real-world scenarios.

Optimizer-Based Strategy

- Plans buying and selling to **maximize profitability**.
- Forecasts 1.5 days ahead; updated daily.
- More efficient battery use and lower degradation.
- Requires a battery to operate in the simulation.

MILP Formulation:

$$\begin{aligned} & \text{minimize} && c^T x \\ & \text{subject to} && 0 \leq s + \sum_{i=1}^t f(x_i + p_i) \leq b_t \\ & && b_{t-1} - d \left(\frac{x_t + p_t}{b_{\text{init}}} \right) = b_t \\ & && -g \leq f(x_t + p_t) \leq g \end{aligned}$$

Variables

$c \in \mathbb{R}^T$: price of energy

$x \in \mathbb{R}^T$: amount of energy bought

$p \in \mathbb{R}^T$: power balance

$b \in \mathbb{R}^T$: battery capacity

$g \in \mathbb{R}$: maximum charge rate

$s \in \mathbb{R}$: starting state of charge

$d \in \mathbb{R}$: degradation per cycle

$f : \mathbb{R} \rightarrow \mathbb{R}$: efficiency function

-
1. Introduction
 2. Basic Implementation
 3. Heatpump
 4. Optimized Strategy
 - Electrical Vehicle
 6. Live Demo
 7. Evaluation
 8. Time Constraints
 9. Future work

Purpose:

Maximize solar self-consumption – Align EV charging with PV generation to reduce grid dependence.

If Optimized:

- Higher PV utilization – Charge EVs during peak solar hours
- Lower electricity bills – Minimize grid imports
- Grid-friendly – Prevent evening demand spikes when PV stops

Core idea:

Use actual driving data (GPS/traffic patterns) to calculate:

$$\text{Energy} = \text{Distance} \times \text{Car Efficiency}$$

Key Features:

- Adapts to seasonal changes (e.g., 20% higher usage in winter)
- Accounts for real-world variability (traffic, detours)

Core Formula:

$$\text{Charging Need} = \frac{\text{User's Annual km}}{365} \times \text{kWh/km}$$

Steps:

- User inputs car model + annual km
- System scales to their driving habits
- Requests energy in chosen time slot

-
1. Introduction
 2. Basic Implementation
 3. Heatpump
 4. Optimized Strategy
 5. Electrical Vehicle
 - Live Demo
 7. Evaluation
 8. Time Constraints
 9. Future work

Live Demo

Optimization of Photovoltaic and Battery Storage Systems

Demo Components

- **AnyLogic Simulation Interface**
- **Performance Optimization Showcase**
- **Interactive Parameter Adjustment**

-
1. Introduction
 2. Basic Implementation
 3. Heatpump
 4. Optimized Strategy
 5. Electrical Vehicle
 6. Live Demo
 - Evaluation
 8. Time Constraints
 9. Future work

Key Mathematical Formulations

Index Decoding:

$$\begin{aligned}\text{battery_count} &= \lfloor \text{index} / 32 \rfloor \\ \text{remaining} &= \text{index} \bmod 32 \\ \text{pv_count} &= \lfloor \text{remaining} / 2 \rfloor \\ \text{strategy} &= \text{remaining} \bmod 2\end{aligned}$$

System Cost Calculation:

$$\begin{aligned}c_{\text{PV}}(\text{kWp}) &= \begin{cases} 0 & \text{if kWp} = 0 \\ 900 \frac{\text{€}}{\text{kWp}} \times \text{kWp} + 4500\text{€} & \text{otherwise} \end{cases} \\ c_{\text{battery}} &= 2500\text{€} \times \text{battery_count} \\ c_{\text{total}} &= c_{\text{battery}} + c_{\text{PV}}(0.88 \times \text{pv_count})\end{aligned}$$

Normalized Financial Return

Value Gain:

Let m = money saved on electricity, $d_{\text{battery}} = 0.4$ (battery depreciation), $d_{\text{PV}} = 0.2$ (PV depreciation)

$$\text{value gain} = m - (d_{\text{battery}} \times c_{\text{battery}} + d_{\text{PV}} \times c_{\text{PV}})$$

Normalized Investment Return:

$$R_{\text{normalized}} = \frac{m}{c_{\text{total}}}$$

Value Gain per Battery: Let b be the battery count

$$\text{value gain per battery} = \frac{m_b - m_0}{b}$$

Money saved on electricity after 8 years

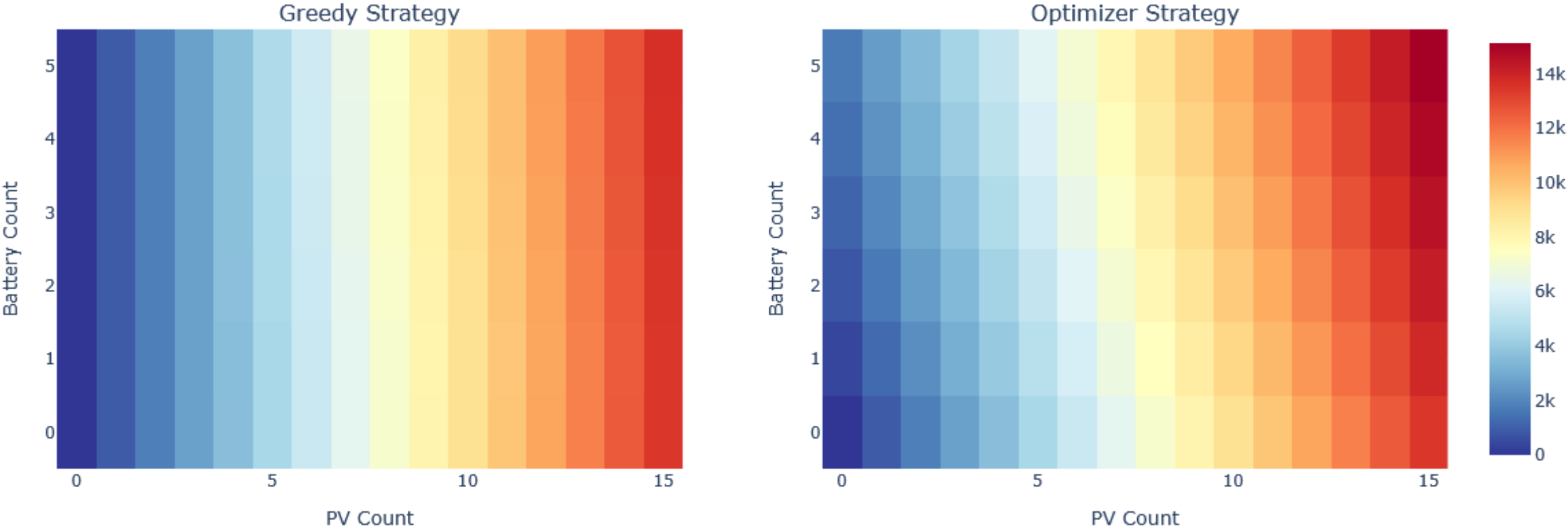


Figure: Money saved on electricity, own depiction

Value gained after 8 years

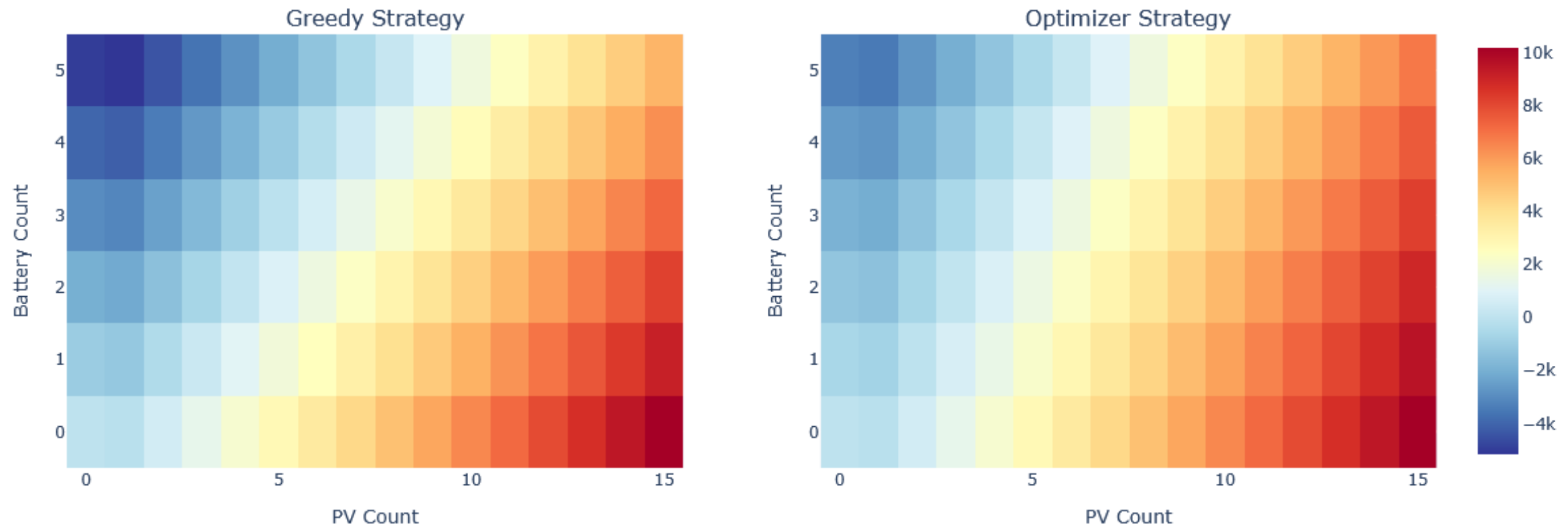


Figure: Total value gained, own depiction

Percentage of investment regained after 8 years

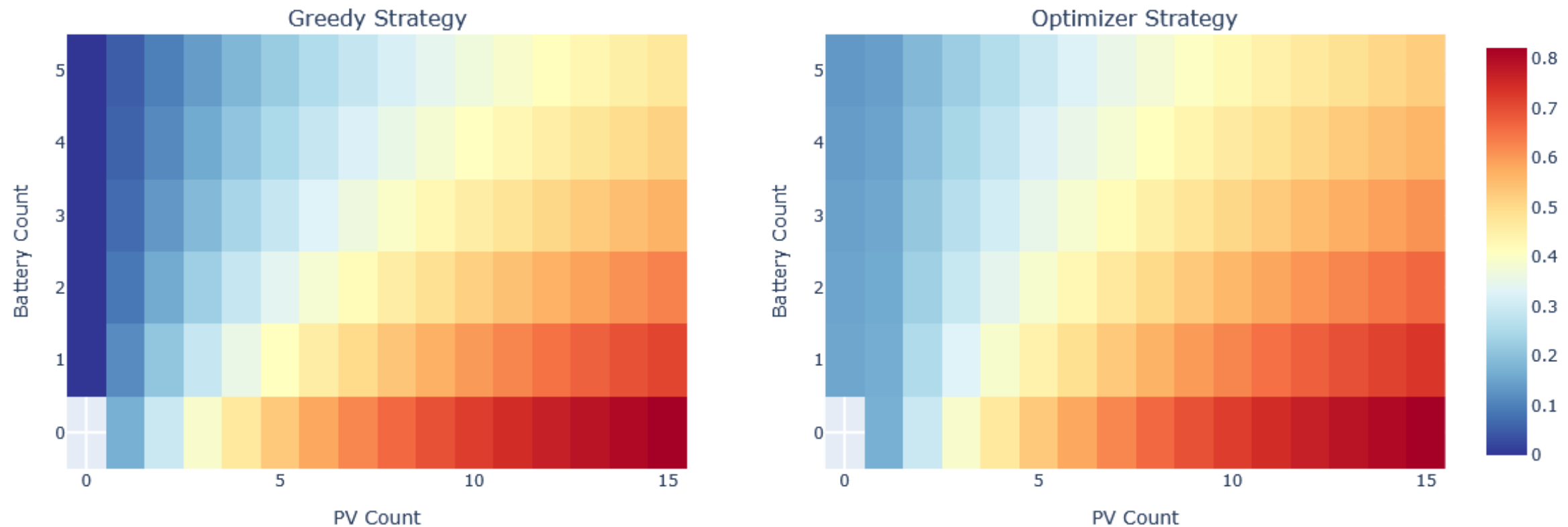


Figure: Percentage regained, own depiction

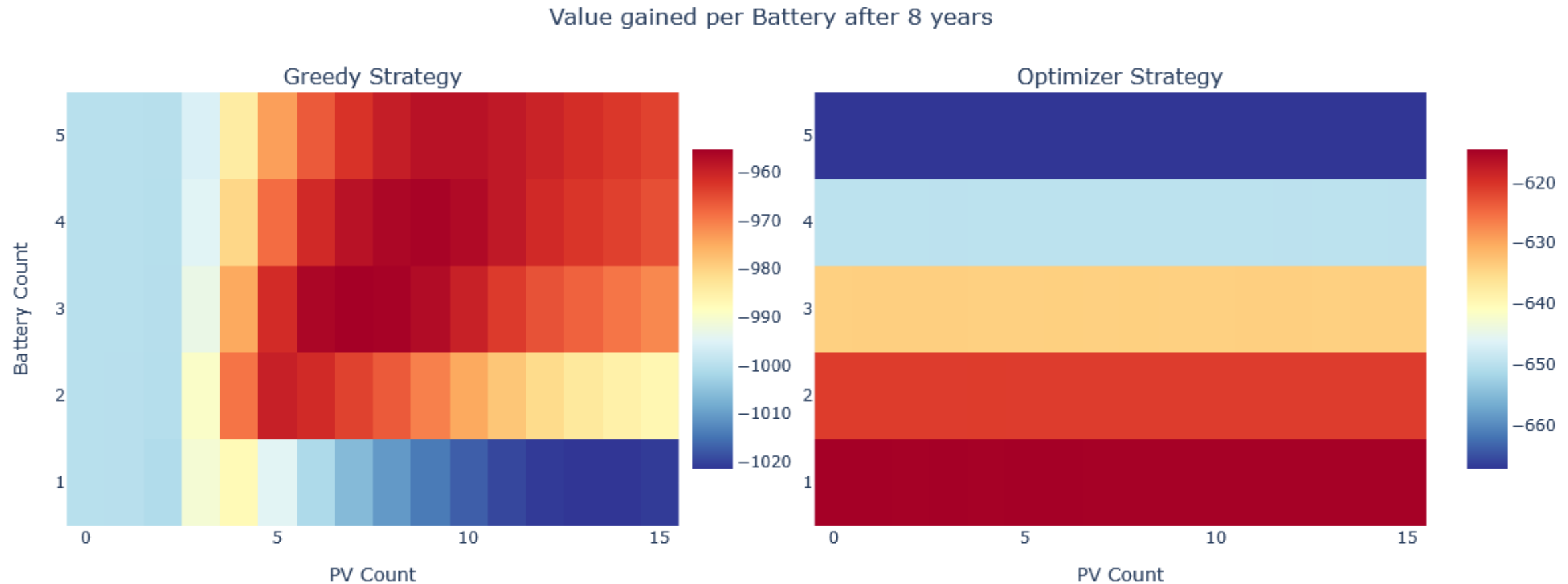
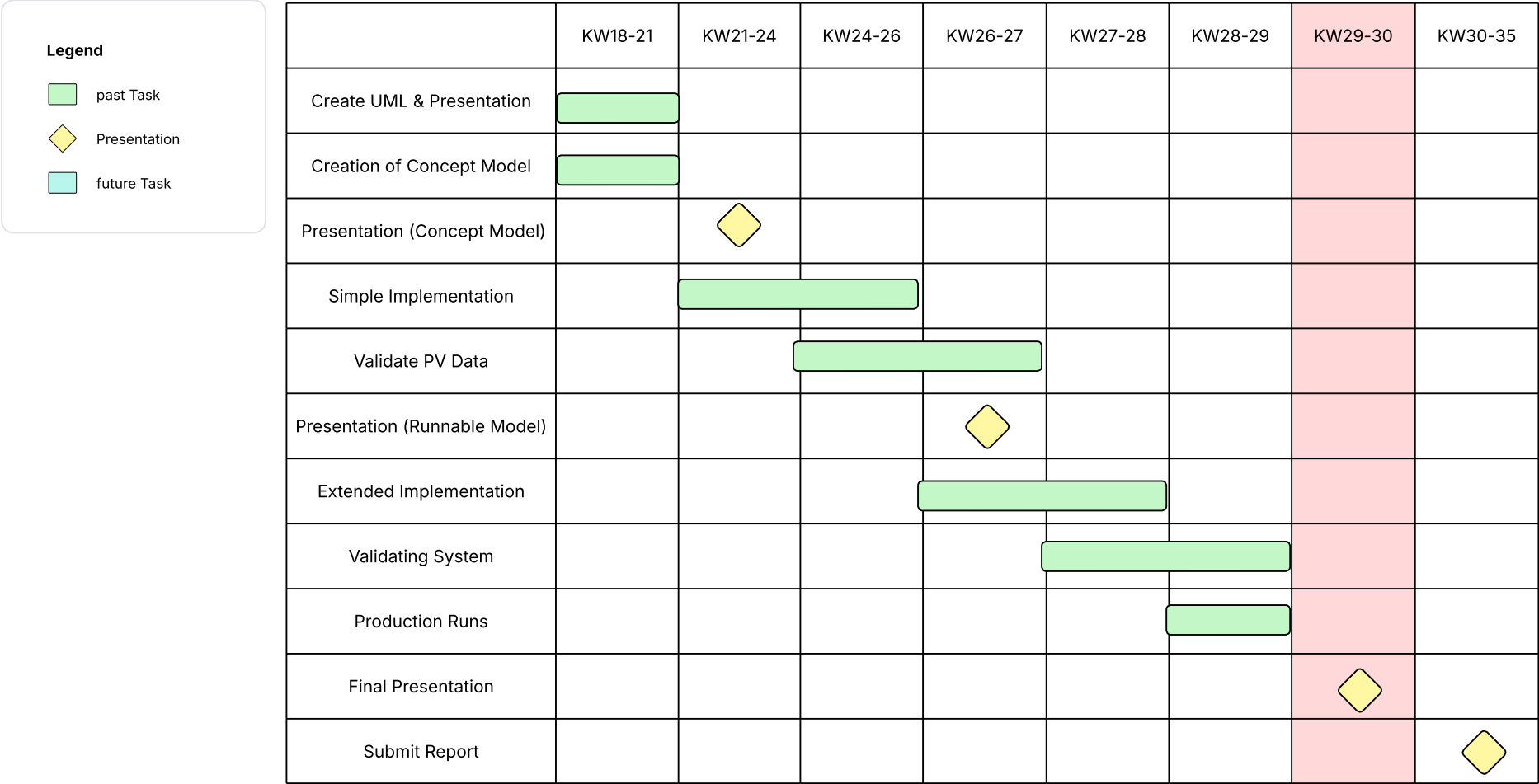


Figure: Value gained per battery

-
1. Introduction
 2. Basic Implementation
 3. Heatpump
 4. Optimized Strategy
 5. Electrical Vehicle
 6. Live Demo
 7. Evaluation
 - Time Constraints
 9. Future work



Database

ArrayOutOfBounds Error
Slow queries → In Memory Caching

Heatpump

No public datasets
Relies on other models

Electric Vehicle

No relevant public datasets

-
1. Introduction
 2. Basic Implementation
 3. Heatpump
 4. Optimized Strategy
 5. Electrical Vehicle
 6. Live Demo
 7. Evaluation
 8. Time Constraints
- Future work

Improvements:

- non perfect future estimation
- more complex Optimizer
- increase search space resolution
- extend simulated timeframe
- increase the timeframe the optimizer uses
 - quite heavy runtime burden for all above
 - improvements in runtime needed
- use NoSQL Database
- model degregation and other parts more realistic

Extensions:

- enable EV implementation with real world data
- optimize EV loading times
- simulate and analyze different scenarios (HP / contract / base parameters)
- add more factors for the individual systems like roof angle for PV

Rethinking assumptions:

- model real world energy selling constraints
- flexible household location
- most data from 2016, equipment assumptions from 2025

Thank You for Your Attention!

We are happy to answer your questions.
Let's discuss and improve together!

Presented by:

Christian Besold, Jakob Haberstock, Nandini Joshi, Robert Leichtl, Florian Merlau



Summary (under our assumptions)

- Batteries are not profitable
- buying more PV modules is advised
- its highly likely that the investment in PV is profitable over its lifetime
- the optimizer doesn't compensate enough for the cost of the batteries for them to be profitable